Low-Level Design (LLD)

Professional-Skepticism Engine — module, schema & agent design

Project	Professional-Skepticism Engine	**Document**	Low-Level Design (LLD)
Version	1.2	**Status**	Reviewed — CSV intake + per-agent I/O worked
Owner	Athithiyan A Aravinth Jay	**Date**	2026-06-26
Pattern	Multi-agent (debate + verify)	**Stack**	LangGraph · Claude · SQLite · RAG

v1.2 changes. Adds the **intake module** (§2.1, §3.1) — CSV ingestion, the typed `Transaction`, and the rule pre-filter — and a new **§12 Per-Agent I/O Contracts** that traces two cases (auto-clear + escalated) showing the concrete input each agent receives and the structured output it returns.

1. Introduction

This LLD specifies the implementation-level design: repository layout, data models, per-agent contracts (model, temperature, prompt role, I/O schema, logic), the LangGraph wiring & routing, the RAG and connector designs, the database schema, configuration, error handling, the test design, and fully-worked per-agent input/output for two cases.

2. Module Decomposition

```
skeptic-engine/
  app/
    state.py        # InvestigationState (TypedDict) + Pydantic models
    graph.py        # LangGraph nodes, edges, routing, checkpointer
    config.py       # thresholds, materiality, model map, round caps
    llm.py          # Claude client + structured-output helper + retry
    db.py           # SQLite: cases, reviews, audit_log (+ hash chain)
  intake/
    ingest.py       # read GL CSV -> rows
    normalize.py    # row -> typed Transaction (Pydantic)
    rules.py        # rule pre-filter -> flags[]; create_case()
  agents/
    supervisor.py evidence.py challenger.py defender.py
    adjudicator.py verifier.py review.py report.py
  rag/
    ingest.py store.py retrieve.py    # chunk -> embed -> hybrid search
  connectors/
    base.py fx.py registry.py cache.py
  ui/  policies/  data/  tests/
  Dockerfile requirements.txt README.md
```

2.1 Intake module logic (intake/)

```
def run_intake(csv_path) -> list[Case]:
    rows = ingest.read_csv(csv_path)              # pandas / csv.DictReader
    cases = []
    for row in rows:
```

```
        txn  = normalize.to_transaction(row)      # typed + validated
        flags = rules.evaluate(txn)               # list[str], may be empty
        if not flags:
            db.record_cleared_at_intake(txn)      # clean row -> dropped, never hits the crew
            continue
        case = create_case(txn, flags)            # seed InvestigationState
        db.save_case(case); cases.append(case)
    return cases                                   # only flagged rows
```

3. Data Model

3.1 Pydantic models

```
class Transaction(BaseModel):                  # NEW — produced by intake
    txn_id: str; date: date; account: str; vendor: str
    description: str; currency: str
    amount: float; fx_rate_used: float; amount_usd: float
    posted_by: str

class PolicyClause(BaseModel):
    id: str; version: str; text: str

class Evidence(BaseModel):
    policy: list[PolicyClause]                 # id, version, text
    history: dict                              # vendor_prior_12m, account_mean, z_score
    external: dict                             # fx, vendor_master, ...
    citations: list[str]
    api_metadata: list[dict]                   # source, status, latency_ms, cached

class ChallengerView(BaseModel):
    position: Literal["concern"]
    severity: Literal["low","medium","high"]
    claims: list[str]
    red_flags: list[str]

class DefenderView(BaseModel):
    position: Literal["legitimate_possible","insufficient_basis"]
    arguments: list[str]
    conditions: list[str]
    confidence_in_legitimacy: float = Field(ge=0, le=1)

class Adjudication(BaseModel):
    risk_level: Literal["High","Medium","Low"]
    confidence: float = Field(ge=0, le=1)
    disposition: str
    rationale: str
    citations: list[str]

class Verification(BaseModel):
    passed: bool
    checked_claims: int; grounded: int; ungrounded: int
    issues: list[str]
    action: Literal["proceed","revise"]

class ReviewEvent(BaseModel):
    reviewer_id: str
    action: Literal["approve","reject","reassign","escalate"]
    comment: str | None; signature: str; timestamp: str
```

3.2 Shared state

```
class InvestigationState(TypedDict):
    case: dict
    plan: dict
    evidence: Evidence | None
    challenger_view: ChallengerView | None
    defender_view: DefenderView | None
    debate_transcript: Annotated[list[dict], add]
    adjudication: Adjudication | None
    verification: Verification | None
    routing: dict
    review_history: Annotated[list[ReviewEvent], add]
    audit_history: Annotated[list[dict], add]
    round: int                                  # debate/verify round counter (bounded)
    report: str | None
```

4. Agent Detailed Design

Agent (node)	Model · temp	I/O (Pydantic)	Core logic
intake	— (deterministic)	CSV row → Transaction + flags[]	normalize + rule pre-filter; create case or drop
supervisor	sonnet · 0.0	case → plan{}	read flags+amount; set rounds, materiality, specialists, route hint
evidence	sonnet · 0.0 (+tools)	case → Evidence	RAG retrieve + history query + FX/registry call; attach citations + api_metadata
challenger	sonnet · 0.5	case+evidence → ChallengerView	argue concern ONLY; cite red flags from evidence ids
defender	sonnet · 0.5	case+evidence → DefenderView	argue legitimacy ONLY; isolated from challenger output
adjudicator	sonnet · 0.0	evidence+views → Adjudication	weigh both; assign risk+confidence+disposition; must cite
verifier	sonnet · 0.0	adjudication+evidence → Verification	map each claim to a citation; set passed/action
review	human	bundle → ReviewEvent	interrupt(); resume with signed decision
report	haiku/code	all → CaseFile	render MD/HTML/JSON case file

4.1 Node signature & prompt contract (pattern)

```
def challenger(state: InvestigationState) -> dict:
    ev = state["evidence"]
    view = call_structured(
        model=MODEL_MAP["challenger"], temperature=0.5,
        system=PROMPTS["challenger"],   # "You are the skeptic. Argue ONLY the
                                        # case for concern. Cite evidence ids."
        user=render(state["case"], ev),
        schema=ChallengerView, retries=3)
```

```
    return {"challenger_view": view,
            "debate_transcript": [{"role":"challenger","content":view.model_dump()}]}
```

5. Orchestration (graph.py)

```
g = StateGraph(InvestigationState)
for name, fn in NODES: g.add_node(name, fn)
g.add_edge(START, "supervisor"); g.add_edge("supervisor", "evidence")
g.add_edge("evidence", "challenger"); g.add_edge("evidence", "defender")   # parallel
g.add_edge(["challenger","defender"], "adjudicator")                       # fan-in
g.add_edge("adjudicator", "verifier")
g.add_conditional_edges("verifier", route_after_verify,                    # revise loop (bounded)
    {"revise": "adjudicator", "gate": "confidence_gate"})
g.add_conditional_edges("confidence_gate", route_after_gate,
    {"auto": "report", "human": "review"})
g.add_conditional_edges("review", route_after_review,
    {"approve": "report", "reject": "report", "revise": "adjudicator"})
g.add_edge("report", "audit"); g.add_edge("audit", END)
app = g.compile(checkpointer=SqliteSaver.from_conn_string("checkpoints.db"))
```

Note: intake (§2.1) runs *before* the graph and seeds one `thread_id` per case; `app.invoke(case_state, config={"configurable":{"thread_id":tid}})`.

5.1 Routing functions

```
def route_after_verify(s):
    if s["verification"].action == "revise" and s["round"] < MAX_DEBATE_ROUNDS:
        s["round"] += 1; return "revise"
    return "gate"

def route_after_gate(s):
    a = s["adjudication"]; amt = s["case"]["amount_usd"]
    if a.confidence >= AUTO and s["verification"].passed and a.risk_level=="Low" \
       and amt < MATERIALITY: return "auto"
    return "human"
```

5.2 Human-in-the-loop

```
def review(state):
    d = interrupt({"transcript": state["debate_transcript"],
                   "adjudication": state["adjudication"].model_dump()})
    return {"review_history": [ReviewEvent(**d)],
            "audit_history": [audit_event("review", d)]}
# UI resumes: app.invoke(Command(resume=decision), config={"configurable":{"thread_id":tid}})
```

6. RAG Design

- **Ingest:** policies (PDF/MD) → split by clause (~200–400 tokens, 15% overlap) → embed → upsert to Chroma with metadata `{doc, clause_id, version, effective_date}`.
- **Retrieve:** build query from case (account, vendor, amount, flags) → hybrid (BM25 + dense, RRF) → top-k=8 → cross-encoder re-rank → top-n=4.
- **Contract:** `retrieve(query, k=8, n=4) -> list[PolicyClause]` with ids the Adjudicator must cite.

7. Connector Design

```
class Provider(Protocol):
    def fetch(self, case) -> dict: ...            # returns an evidence fragment

class FXProvider:                                 # connectors/fx.py
    def fetch(self, case):
        key = f"{case['currency']}:{case['date']}"
        if key in cache: return cache[key]
        try:
            r = httpx.get(f"https://api.frankfurter.app/{case['date']}",
                          params={"from":case["currency"],"to":"USD"}, timeout=3)
            ev = build(r.json()); cache[key]=ev; return ev
        except Exception:
            return {"status":"unavailable"}        # missing -> lowers confidence
```

8. Database Schema (SQLite)

Table	Columns
cases	case_id PK · txn_id · payload_json · status · created_at
reviews	id PK · case_id FK · reviewer_id · action · comment · signature · ts
audit_log	id PK · case_id · event · payload_json · hash_prev · hash · ts
checkpoints	managed by LangGraph SqliteSaver — thread_id · state blob

`audit_log` is append-only; each row stores `hash = sha256(hash_prev + payload)` for tamper-evidence.

9. Configuration (config.py)

```
AUTO, REVIEW   = 0.90, 0.70
MATERIALITY    = 25_000
MAX_DEBATE_ROUNDS = 2
ROUND_AMOUNT_RULE = lambda a: a % 1000 == 0       # round-number flag
GENERIC_POSTERS   = {"dadmin", "system"}          # SoD flag
MODEL_MAP = {"supervisor":"sonnet","evidence":"sonnet","challenger":"sonnet",
             "defender":"sonnet","adjudicator":"sonnet","verifier":"sonnet",
             "report":"haiku"}
TEMPS = {"challenger":0.5,"defender":0.5,"adjudicator":0.0,"verifier":0.0,"supervisor":0.0}
```

10. Error Handling & Edge Cases

Case	Handling
Malformed CSV row	Pydantic validation error in normalize → row quarantined to `intake_errors`, not fatal
Claude 429 / 5xx / timeout	Exponential backoff retry (3x) in call_structured; then mark node failed → DLQ
External API down	Timeout + cache fallback; record evidence as unavailable; lower confidence
Verifier fails repeatedly	Bounded re-runs (MAX_DEBATE_ROUNDS); then force route = manual

Malformed LLM output	Pydantic validation error → 1 reprompt with the error → else manual
Reviewer never responds	Case stays paused in checkpoint (durable); SLA escalation (future)

11. Test Design

- **Intake:** rule unit tests — assert each rule fires on a crafted row and clean rows produce no flags.
- **Unit (per agent):** mock the LLM (canned structured output), assert schema + routing behaviour.
- **RAG:** RAGAS context precision/recall on the seeded policy set; citation IDs present.
- **Verifier:** inject ungrounded claims → assert `action=="revise"`.
- **Integration:** full case run; assert auto-clear vs human path; restart mid-debate resumes.
- **Eval harness:** RAGAS suite on a labelled golden set → faithfulness, context P/R (PRD §15).

12. Per-Agent I/O Contracts — Two Worked Cases

This section shows the concrete input each agent receives and the structured output it returns, for two cases drawn from `sample_gl_1000.csv`: **Case A** trips a soft rule but is legitimate and **auto-clears**; **Case B** is a related-party anomaly that fails an early verification, re-runs, and **escalates to a partner**. The full narrative report for each is in the Sample Output document.

12.1 Case A — auto-clear · TXN-0003 (Brookfield rent, FX)

PHASE 0 — INTAKE · input (CSV row) → output (Transaction + flags)

```
row  = {"txn_id":"TXN-0003","date":"25-06-2026","account":"Rent","vendor":"Brookfield Properties",
        "description":"Rent expense","currency":"INR","amount":1558481.67,
        "fx_rate_used":0.012,"amount_usd":18701.78,"posted_by":"jsmith"}
# rules.evaluate ->
case = {"case_id":"CASE-0042","txn_id":"TXN-0003", ...,"amount_usd":18701.78,
        "flags":["fx_conversion"], "status":"intake"}    # flagged (non-USD) -> becomes a case
```

1 · SUPERVISOR · input `case` → output `plan`

```
plan = {"steps":["gather_evidence","run_debate","adjudicate","verify"],
        "specialists":["evidence","challenger","defender","adjudicator","verifier"],
        "max_debate_rounds":2, "materiality_usd":25000,
        "escalation":"none (amount < materiality)",
        "route_hint":"soft FX flag -> expect auto-clear if grounded"}
```

2 · EVIDENCE · input `case` → output `evidence`

```
evidence = {
  "policy":[{"id":"LSE-01 §3","v":"2.0","text":"Recorded lease/rent must match the signed lease
schedule."},
            {"id":"FX-04 §1","v":"1.2","text":"Foreign-currency postings use the daily reference
rate ±2%."}],
  "history":{"vendor_prior_12m":11,"account_mean_usd":18650,"z_score":0.04},
  "external":{"fx":{"INR->USD@2026-06-25":0.01201,"posted":0.012,"delta":"0.1% (within ±2%)"},
              "vendor_master":"Brookfield Properties — APPROVED (lease LSE-1188)"},
  "citations":["LSE-01 §3","FX-04 §1","history:Rent","fx:frankfurter","vendormaster"],
  "api_metadata":[{"source":"frankfurter","status":200,"latency_ms":210,"cached":false}]}
```

3 · CHALLENGER · input `case+evidence` → output `challenger_view`

```
challenger_view = {"position":"concern","severity":"low",
  "claims":["Large foreign-currency posting (INR 1.56M) warrants a rate check.",
            "Amount is above the account mean, though only marginally (z=0.04)."],
  "red_flags":["fx_conversion"]}
```

4 · DEFENDER · input `case+evidence` (not the Challenger) → output `defender_view`

```
defender_view = {"position":"legitimate_possible","confidence_in_legitimacy":0.95,
  "arguments":["Vendor is an approved lessor (LSE-1188); 11 matching monthly rents in 12m.",
               "FX rate 0.012 is within 0.1% of the daily reference (FX-04 §1).",
               "amount_usd 18,701.78 sits on the account mean (z=0.04) — not an outlier."],
  "conditions":["lease schedule on file"]}
```

5 · ADJUDICATOR · input `evidence+both views` → output `adjudication`

```
adjudication = {"risk_level":"Low","confidence":0.94,
  "disposition":"clear_recurring_legitimate",
  "rationale":"Approved lessor, recurring identical rent, FX within tolerance, on-mean amount. "
              "The only flag (fx_conversion) is fully explained by grounded evidence.",
  "citations":["LSE-01 §3","FX-04 §1","history:Rent z=0.04","fx:frankfurter:
0.1%","vendormaster:approved"]}
```

6 · VERIFIER · input `adjudication+evidence` → output `verification`

```
verification = {"passed":true,"checked_claims":4,"grounded":4,"ungrounded":0,
  "issues":[], "action":"proceed"}
```

7 · CONFIDENCE GATE · input `adjudication+verification+amount` → output `routing`

```
routing = {"decision":"auto",
  "reason":"confidence 0.94 ≥ 0.90 AND verified AND risk=Low AND amount 18,701.78 < 25,000"}
# no human review; goes straight to report (QA-sampled)
```

8 · REPORT / AUDIT · input `full state` → output `case file + audit entry`

```
audit_history += {"event":"case_closed","case_id":"CASE-0042",
  "verdict":"Low / clear_recurring_legitimate","confidence":0.94,
  "route":"auto_cleared","human":null,
  "sources_used":["LSE-01 §3","FX-04 §1","fx:frankfurter","vendor_master"],
  "timestamp":"2026-06-26T09:14:02Z","hash_prev":"…","hash":"…"}
```

12.2 Case B — escalated · TXN-0876 (related-party, fails verify once)

PHASE 0 — INTAKE · input (CSV row) → output (Transaction + flags)

```
case = {"case_id":"CASE-0007","txn_id":"TXN-0876","date":"2026-05-20","account":"Consulting",
        "vendor":"Mehta Family Holdings","description":"Consulting - related party (director J.
Mehta)",
        "currency":"USD","amount":58000,"amount_usd":58000,"posted_by":"rkumar",
        "flags":["materiality","round-dollar amount","statistical outlier (Consulting)",
                 "related-party indicator","unknown vendor"], "status":"intake"}
```

1 · SUPERVISOR · output `plan`

```
plan = {"steps":["gather_evidence","run_debate","adjudicate","verify"],
        "specialists":["evidence","challenger","defender","adjudicator","verifier"],
        "max_debate_rounds":2,"materiality_usd":25000,
```

```
        "escalation":"partner_if_material",
        "route_hint":"material + related-party -> expect manual / partner"}
```

2 · EVIDENCE · output `evidence`

```
evidence = {
  "policy":[{"id":"RP-01 §2","v":"2.1","text":"Related-party txns require independent review (ISA
550)."},
            {"id":"AP-02 §1","v":"3.0","text":">$5k needs manager approval; >$25k needs partner
approval."}],
  "history":{"vendor_prior_12m":0,"account_mean_usd":15126,"z_score":4.1},
  "external":{"fx":"n/a (USD)","vendor_master":"Mehta Family Holdings — NOT in approved vendor
master"},
  "citations":["RP-01 §2","AP-02 §1","history:Consulting","vendormaster"],
  "api_metadata":[{"source":"vendor_master","status":200,"cached":false}]}
```

3 · CHALLENGER · output `challenger_view`

```
challenger_view = {"position":"concern","severity":"high",
  "claims":["Vendor implies related party to director J. Mehta (RP-01 §2).",
            "$58,000 is 4.1σ above the Consulting mean AND an exact round number.",
            "Vendor absent from approved master; no 12-month history (ghost vendor).",
            "$58,000 exceeds the $25,000 partner-approval threshold (AP-02 §1)."],
  "red_flags":["related_party","round_dollar","outlier","ghost_vendor","approval_breach"]}
```

4 · DEFENDER · output `defender_view`

```
defender_view = {"position":"legitimate_possible","confidence_in_legitimacy":0.25,
  "arguments":["Large one-off strategy retainers occur in Consulting; can be round by contract.",
               "Related-party is permitted IF independently approved and disclosed.",
               "A new vendor naturally has no 12-month history."],
  "conditions":["signed contract","independent approval","related-party disclosure on file"]}
```

5 · ADJUDICATOR (round 1) · output `adjudication`

```
adjudication = {"risk_level":"High","confidence":0.86,
  "disposition":"investigate_before_reliance",
  "rationale":"Related-party + 4.1σ round-number outlier + ghost vendor + breach of AP-02 "
             "outweighs the benign-retainer hypothesis, which has no approval/disclosure on file. "
             "Director sign-off was obtained on 2026-05-19.",   # <-- unsupported claim
  "citations":["RP-01 §2","AP-02 §1","history:Consulting z=4.1","vendormaster:absent"]}
```

6 · VERIFIER (round 1) · output `verification` → FAIL → revise

```
verification = {"passed":false,"checked_claims":5,"grounded":4,"ungrounded":1,
  "issues":["'Director sign-off obtained 2026-05-19' has NO supporting evidence id
(hallucination)."],
  "action":"revise"}
# route_after_verify: round 0->1, < MAX_DEBATE_ROUNDS -> loop back to adjudicator
```

5' · ADJUDICATOR (round 2, revised) · output `adjudication`

```
adjudication = {"risk_level":"High","confidence":0.86,
  "disposition":"investigate_before_reliance",
  "rationale":"Related-party + 4.1σ round-number outlier + ghost vendor + breach of AP-02 "
             "partner-approval threshold; NO approval or disclosure evidence on file.",
  "citations":["RP-01 §2","AP-02 §1","history:Consulting z=4.1","vendormaster:absent"]}
```

6' · VERIFIER (round 2) · output `verification` → PASS

```
verification = {"passed":true,"checked_claims":4,"grounded":4,"ungrounded":0,
  "issues":[], "action":"proceed"}
```

7 · CONFIDENCE GATE · output `routing`

```
routing = {"decision":"manual",
  "reason":"amount $58,000 ≥ materiality $25,000 AND risk=High -> partner escalation"}
```

8 · HUMAN REVIEW · input `bundle+transcript` → output `review`

```
review = {"reviewer_id":"a.sharma (Partner)","action":"reject",
  "comment":"Hold payment. Obtain signed contract + independent approval + related-party "
            "disclosure before any reliance.",
  "signature":"sha256:9f2c4b…e71","timestamp":"2026-06-25T11:42:08Z"}
```

9 · REPORT / AUDIT · output `audit entry`

```
audit_history += {"event":"case_closed","case_id":"CASE-0007",
  "verdict":"High / investigate_before_reliance","confidence":0.86,
  "rounds":2,"route":"manual","escalated_to":"partner",
  "human":{"reviewer":"a.sharma(Partner)","action":"reject","signature":"sha256:9f2c4b…e71"},
  "sources_used":["RP-01 §2","AP-02 §1","vendor_master","history:Consulting"],
  "timestamp":"2026-06-25T11:42:09Z","hash_prev":"…","hash":"…"}
```

Contrast. Case A trips one soft flag, is fully grounded, and the gate sends it to `auto-clear` with no human. Case B trips five flags, the Verifier **catches a hallucinated approval** and forces a re-run, and the gate sends the High/material verdict to `partner escalation`. The two paths exercise every branch in §5.